OPERATING SYSTEMS · RESEARCH ARTICLE

A Comparative Study of Process Synchronization Algorithms

Dekker · Peterson · Semaphore · Monitor · Readers–Writers

Article title	Comparison of Process Synchronization Algorithms
Author	Mohammadreza Azodi
Instructor	Majid Shamshirgaran
Course	Embedding Systems
University / Faculty	Wshragh University
Date	November ١٠, ٢٠٢٦
Language	English

Table of Contents

Abstract ٣
١. Introduction and Fundamentals ٤
The Critical-Section Problem ٤
Race Conditions: Why Synchronization Is Needed ٤
The Three Requirements of a Correct Solution ٤
Comparison Criteria ٥
٢. Dekker's Algorithm ٦
History and Idea ٦
Pseudocode ٦
C Implementation ٦
Correctness Analysis ٧
Strengths and Weaknesses ٨
٣. Peterson's Algorithm ٩
History and Idea ٩
Pseudocode ٩
C Implementation ٩
Execution Timeline ١٠
Correctness Analysis ١٠
Strengths and Weaknesses ١١
٤. Semaphores ١٢
Definition and Idea ١٢
Pseudocode ١٢
C Implementation ١٣
Analysis and Pitfalls ١٤
٥. Monitors ١٦
Definition and Idea ١٦
Two Signalling Styles: Hoare and Mesa ١٦
Pseudocode ١٦
C and Java Implementations ١٦
Analysis ١٨
٦. Readers–Writers ١٩
Problem Statement ١٩
The Three Variants ١٩
Pseudocode ١٩
C Implementation ٢٠
Timeline and Starvation ٢١
Analysis ٢١

٧. Comprehensive Comparison ٢٣
Correctness and Behaviour ٢٣
Engineering and Efficiency ٢٣
Qualitative Scoring ٢٣
٨. Worked Examples and Sample Questions ٢٦
Dekker ٢٦
Peterson ٢٦
Semaphore ٢٧
Monitor ٢٨
Readers–Writers ٢٨
٩. Conclusion and Selection Guide ٣٠
References ٣١

To refresh page numbers in Word: right-click the table and choose Update Field (or press F٩).

Abstract

Process synchronization is one of the cornerstones of operating systems and concurrent programming. Whenever several processes access a shared resource and at least one of them modifies it, the result depends on the scheduler's timing unless the accesses are carefully coordinated. This article studies and compares five classical solutions: **Dekker's algorithm**, **Peterson's algorithm**, **semaphores**, **monitors** and the **Readers–Writers** problem.

For each solution we give the main idea, pseudocode, a C implementation (Java for monitors), a flowchart, a correctness analysis and its strengths and weaknesses. All solutions are then compared in tables and charts using mutual exclusion, progress, bounded waiting, busy waiting, scalability, deadlock, starvation and code complexity. Worked examples and sample questions with answers close the article. All C listings were compiled and tested.

Keywords
Synchronization, critical section, mutual exclusion, Dekker, Peterson, semaphore, monitor, readers–writers

١. Introduction and Fundamentals

The Critical-Section Problem

Assume several processes (or threads) run concurrently and all access shared data such as a counter, a buffer or a file. The part of each process's code that touches this shared data is its **critical section**. The goal is that at most one process is inside its critical section at any moment. As Figure ١ shows, every process consists of an entry section, the critical section, an exit section and a remainder section.

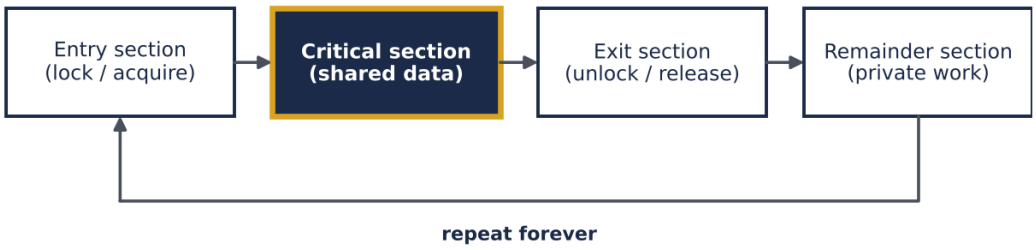


Figure 1 — General structure of a process in the critical-section problem

Race Conditions: Why Synchronization Is Needed

The simple statement `counter++` is not one machine action. It consists of three steps: load the value, add one, and store the result. If two threads interleave these steps, one update is lost. Table ١ shows this for an initial value of `counter = 5`: after two increments the expected value is ٧, but the result is ٦.

Table ١ — An interleaving of two threads that loses an update (initial value ٥)

Step	Thread ١	Thread ٢	Memory	Registers
١	R١ = counter	—	٥	R١=٥
٢	—	R٢ = counter	٥	R١=٥, R٢=٥
٣	R١ = R١ + ١	—	٥	R١=٦, R٢=٥
٤	—	R٢ = R٢ + ١	٥	R١=٦, R٢=٦
٥	counter = R١	—	٦	—
٦	—	counter = R٢	٦ (expected ٧)	—

The Three Requirements of a Correct Solution

Every solution must satisfy three classical requirements, which are the main yardsticks of the later comparison: **mutual exclusion** (no two processes inside their critical sections at the same time), **progress** (when the critical section is free, the choice of the next process cannot be postponed indefinitely and involves only processes that want to enter) and **bounded waiting** (there is a bound on how many times others may enter after a process has requested entry, i.e. no starvation).

Comparison Criteria

The algorithms are judged by: (١) mutual exclusion, (٢) progress, (٣) bounded waiting, (٤) busy waiting and CPU use, (٥) number of supported processes and scalability, (٦) hardware/OS support needed, (٧) deadlock and starvation, (٨) code size and complexity, (٩) ease of use and verification, and (١٠) practical use.

Table ٢ — Overview of the five studied solutions

Solution	Level	Year / inventor	Nature	Core idea
Dekker	Software	١٩٦٥ · T. J. Dekker (published by Dijkstra)	Algorithm, ٢ processes	flag + turn, back-off
Peterson	Software	١٩٨١ · G. L. Peterson	Algorithm, ٢ processes	flag + victim (turn)
Semaphore	OS primitive	١٩٦٥ · E. W. Dijkstra	Synchronization primitive	integer + P / V + queue
Monitor	Language construct	١٩٧٤ · C. A. R. Hoare	Abstract data type	mutex + condition variables
Readers–Writers	Problem / pattern	١٩٧١ · Courtois, Heymans, Parnas	Synchronization problem	shared reads, exclusive writes

٢. Dekker's Algorithm

History and Idea

Dekker's algorithm is the first correct software solution to the mutual-exclusion problem for two processes. It was devised by the Dutch mathematician T. J. Dekker around ١٩٦٠ and published and generalised to N processes by E. W. Dijkstra in ١٩٦٥. Its historical value is that it needs no special hardware instruction such as Test-and-Set — only reads and writes of shared variables.

The idea combines two mechanisms. First, each process raises a flag (`wants[i]`) to announce its interest. Second, the variable `turn` decides who has priority when both flags are up. The process without priority temporarily lowers its flag (**back-off**), waits until it is its turn and raises the flag again. This back-off separates the algorithm from naive flag-only solutions that deadlock.

Pseudocode

Pseudocode ١ — Dekker's algorithm for process i (j = ١ - i)

```
 1  shared: boolean wants[2] = {false, false}
 2          int turn = 0                      // 0 or 1
 3
 4  process P(i):                             // j = 1 - i
 5    loop forever
 6      wants[i] <- true                      // announce interest
 7      while wants[j] do
 8        if turn != i then                   // the other has priority
 9          wants[i] <- false                 // back off
10          wait until turn == i
11          wants[i] <- true
12      end while
13      ----- CRITICAL SECTION -----
14      turn <- j                             // give priority away
15      wants[i] <- false
16      ----- REMAINDER SECTION -----
17    end loop
```

C Implementation

The code uses standard C١١ atomics. Using `atomic_int` (default `seq_cst` ordering) is essential: on modern processors, without memory barriers the CPU or compiler may reorder reads and writes and break the algorithm. The code was tested with two threads and a shared counter, which always ended with the correct value.

Listing \ — Dekker lock (C\ \)

```
#include <stdatomic.h>

atomic_int wants[2];          /* wants[i] = 1 : process i wants to enter */
atomic_int turn = 0;          /* who has priority when both want to enter */

void dekker_lock(int i) {
    int j = 1 - i;
    atomic_store(&wants[i], 1);
    while (atomic_load(&wants[j])) {          /* the other one wants too */
        if (atomic_load(&turn) != i) {        /* not my turn: back off   */
            atomic_store(&wants[i], 0);
            while (atomic_load(&turn) != i)   /* busy wait for my turn   */
                ;
            atomic_store(&wants[i], 1);
        }
    }
}

void dekker_unlock(int i) {
    atomic_store(&turn, 1 - i);               /* hand priority over      */
    atomic_store(&wants[i], 0);
}
```

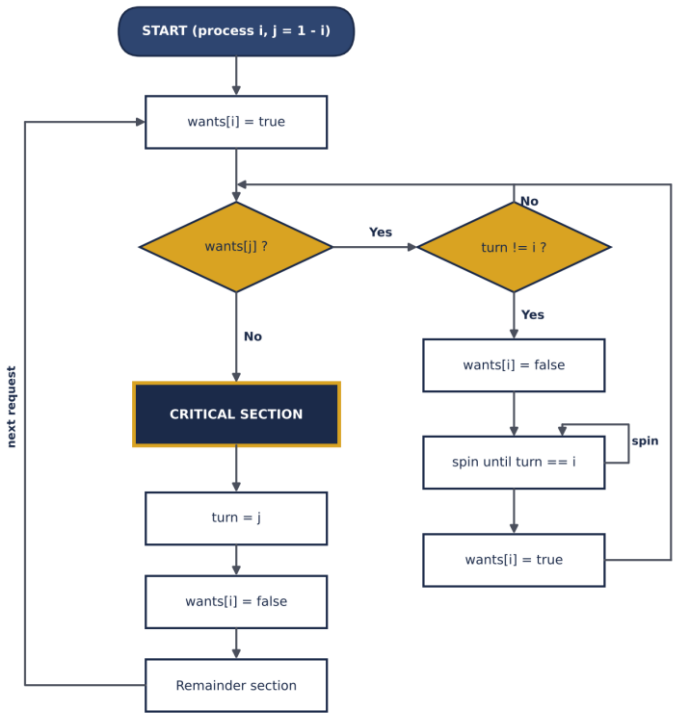


Figure 2 — Flowchart of Dekker's algorithm (process i)

Correctness Analysis

Mutual exclusion: to enter, process i must see `wants[j] = false`, but j raised its flag before checking `wants[i]`, so at least one process sees the other's flag and both cannot enter. **Progress:** if both want to enter, `turn` picks a winner; the loser backs off, so there is no deadlock. **Bounded waiting:** a leaving process hands `turn` to its rival, so the rival waits at most one critical section.

Strengths and Weaknesses

Aspect	Strengths	Weaknesses
Hardware	No special instruction needed	Needs sequentially consistent memory (barriers)
CPU	Simple, tiny memory (٣ variables)	Busy waiting wastes CPU cycles
Processes	Correct and starvation-free for ٢	Only ٢ processes; N-process version is complex
Clarity	Historically important	Hard to understand and to prove; rarely used in practice

٣. Peterson's Algorithm

History and Idea

In ١٩٨١ Gary L. Peterson published a short paper, "Myths about the Mutual Exclusion Problem", with an algorithm far simpler than Dekker's. It also needs only reads and writes of shared variables, but its logic is shorter and its correctness proof easier, which is why textbooks use it as the standard software algorithm.

Each process first announces interest (`flag[i] = true`) and then **gives the turn to its rival** (`turn = j`). It waits only while the rival is interested and the turn is the rival's. If both arrive simultaneously, the process that writes `turn` last has given the turn away and waits, so exactly one passes.

Pseudocode

Pseudocode ٢ — Peterson's algorithm for process i (j = ١ - i)

```
shared: boolean flag[2] = {false, false}
        int turn                          // the process that must wait

process P(i):                             // j = 1 - i
  loop forever
    flag[i] <- true                       // I am interested
    turn    <- j                          // let the other go first
    while flag[j] and turn = j do
      ;                                   // busy wait
    ----- CRITICAL SECTION -----
    flag[i] <- false                      // I am done
    ----- REMAINDER SECTION -----
  end loop
```

C Implementation

Peterson's lock has only twelve effective lines of code. As with Dekker's algorithm, C١١ atomics must be used. The order of the two writes in the entry section (`flag[i]` first, then `turn`) is crucial: swapping them breaks mutual exclusion.

Listing ٢ — Peterson lock (C١١)

```c
#include <stdatomic.h>

atomic_int flag[2];           /* flag[i] = 1 : process i is interested */
atomic_int turn = 0;          /* the process that must wait            */

void peterson_lock(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], 1);        /* I am interested               */
    atomic_store(&turn, j);           /* politely give the turn away   */
    while (atomic_load(&flag[j]) && atomic_load(&turn) == j)
        ;                             /* busy wait                     */
}

void peterson_unlock(int i) {
    atomic_store(&flag[i], 0);
}
```

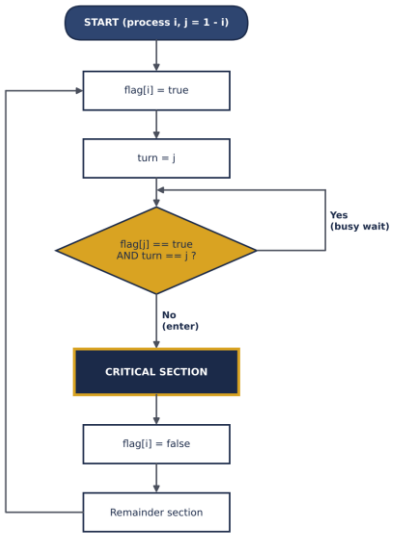


Figure 3 — Flowchart of Peterson's algorithm (process i)

Execution Timeline

Figure ٤ shows a sample run. P٠ requests entry at time ١ and P١ at ١٫٥. P٠ arrives first and takes the critical section while P١ busy-waits until time ٥. In the second round both arrive almost together; since P١ is the last writer of `turn`, it has given the turn to P٠ and waits.

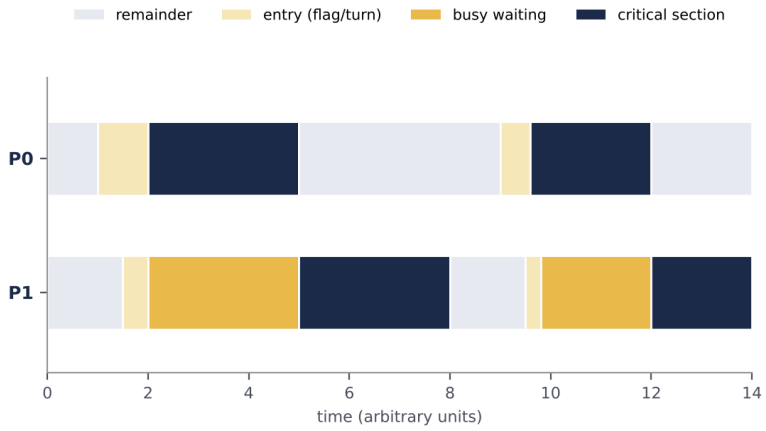


Figure 4 — Timeline of two processes under Peterson's algorithm

Correctness Analysis

Mutual exclusion: suppose both are inside. Each must have seen the loop condition false: P٠ saw `flag[1]=false` or `turn=0`, and P١ saw `flag[0]=false` or `turn=1`. Both raised their flags before writing `turn`, so only `turn` being ٠ and ١ at once remains, which is impossible. **Progress:** a process in its remainder section has a lowered flag and never blocks the other. **Bounded waiting:** if the rival re-requests immediately, it writes `turn` in favour of the waiting process, so the wait is at most one critical section.

> **Practical warning**
> On multicore CPUs without memory barriers (or with plain volatile variables) Peterson's algorithm can fail. Also, if the thread holding the critical section is preempted, its rival burns its whole time slice busy-waiting — we observed this in our own tests on a single CPU core.

Strengths and Weaknesses

Aspect	Strengths	Weaknesses
Simplicity	Very short, easy to prove correct	Limited to ٢ processes
Fairness	Strict bounded waiting (at most ١ turn)	Busy waiting wastes CPU
Hardware	Only atomic loads and stores	Fails without memory barriers on modern CPUs
N processes	Filter lock generalisation exists	Filter lock needs N-١ levels and ٢N-١ variables

٤. Semaphores

Definition and Idea

Semaphores were introduced by Edsger W. Dijkstra in ١٩٦٥. A semaphore is an integer variable modified only through two **atomic** operations: `wait(S)` (originally P, from Dutch *proberen*, "to test") and `signal(S)` (originally V, *verhogen*, "to increment"). `wait` decrements the value and blocks the process if no resource is left; `signal` increments it and wakes a waiting process if there is one. Unlike Dekker and Peterson, a waiting process releases the CPU and sleeps.

Types

Binary (mutex): value ٠ or ١, used for mutual exclusion. **Counting**: any non-negative value, used to manage N instances of a resource (e.g. N free buffer slots). **Strong vs weak**: a strong semaphore has a FIFO wait queue and guarantees bounded waiting; a weak one wakes processes in unspecified order, so starvation is possible.

Pseudocode

Pseudocode ٣ — Blocking semaphore implementation

```
type semaphore = record
    int  value
    list queue                    // processes blocked on this semaphore
end record

wait(S):                          // executed atomically
  S.value <- S.value - 1
  if S.value < 0 then
    append this process to S.queue
    block()                       // sleep, CPU is released

signal(S):                        // executed atomically
  S.value <- S.value + 1
  if S.value <= 0 then
    remove a process Q from S.queue
    wakeup(Q)
```

Pseudocode ٤ — Mutual exclusion and producer–consumer

```
// (a) mutual exclusion with a binary semaphore
semaphore mutex = 1
process P(i):
  loop forever
    wait(mutex)
    ----- CRITICAL SECTION -----
    signal(mutex)
    ----- REMAINDER SECTION -----

// (b) bounded buffer with capacity N
semaphore empty = N, full = 0, mutex = 1

producer:                         consumer:
  loop forever                      loop forever
    item <- produce()                 wait(full)
    wait(empty)                       wait(mutex)
    wait(mutex)                       item <- buffer.remove()
    buffer.insert(item)               signal(mutex)
    signal(mutex)                     signal(empty)
    signal(full)                      consume(item)
```

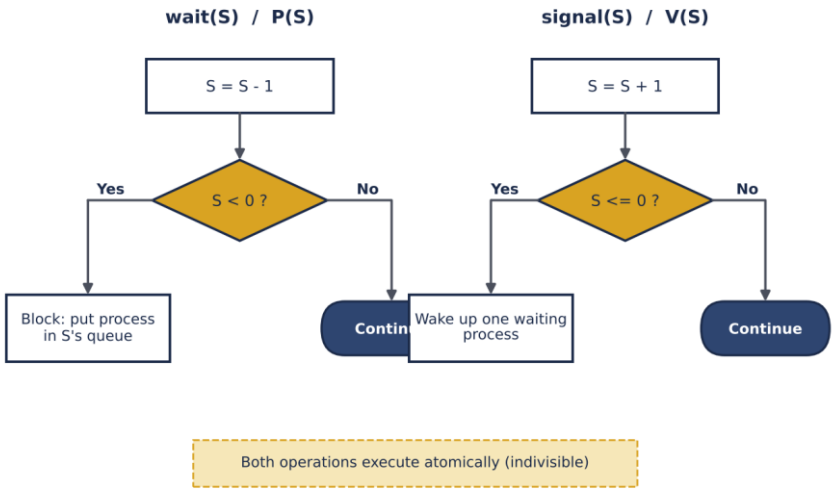


Figure 5 — wait and signal operations of a blocking semaphore

C Implementation

The code solves the bounded-buffer problem with POSIX semaphores. `empty` counts free slots (the producer waits on it), `full` counts filled slots (the consumer waits on it) and `mutex` protects the buffer indices. In our test one producer generated ١٠٠,٠٠٠ numbers, one consumer consumed them all, and the sum came out exactly right.

Listing ٣ — Bounded buffer with POSIX semaphores

```
#include <semaphore.h>

#define N 8                       /* buffer capacity */
int buf[N], in = 0, out = 0;

sem_t empty_slots;                /* init: N  (free slots)  */
sem_t full_slots;                 /* init: 0  (filled slots) */
sem_t mutex;                      /* init: 1  (binary semaphore) */

void init(void) {
    sem_init(&empty_slots, 0, N);
    sem_init(&full_slots,  0, 0);
    sem_init(&mutex,       0, 1);
}

void produce(int item) {
    sem_wait(&empty_slots);       /* P(empty) */
    sem_wait(&mutex);             /* P(mutex) */
    buf[in] = item;  in = (in + 1) % N;
    sem_post(&mutex);             /* V(mutex) */
    sem_post(&full_slots);        /* V(full)  */
}

int consume(void) {
    int item;
    sem_wait(&full_slots);        /* P(full)  */
    sem_wait(&mutex);             /* P(mutex) */
    item = buf[out];  out = (out + 1) % N;
    sem_post(&mutex);             /* V(mutex) */
    sem_post(&empty_slots);       /* V(empty) */
    return item;
}
```

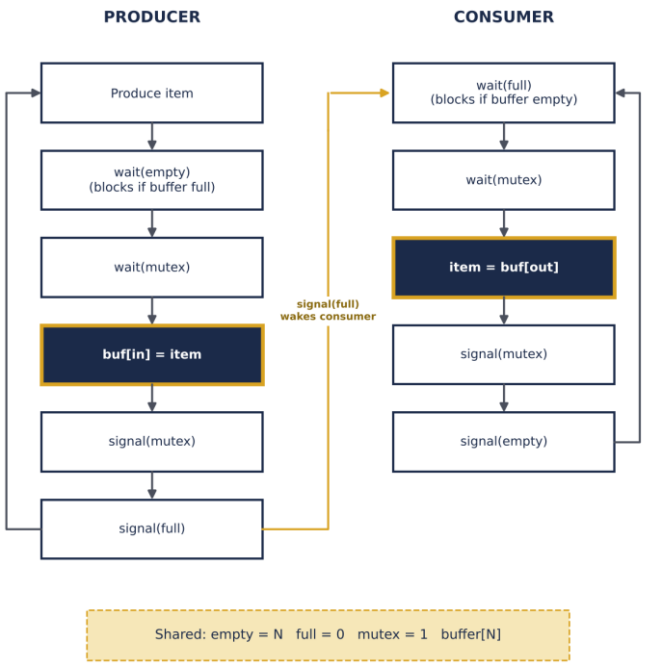


Figure 6 — Producer–consumer flowchart with semaphores

Figure ٧ shows the semaphore values for a buffer of capacity ٣ and ten consecutive operations. `empty + full` always equals ٣. After step ٣ the buffer is full (`empty = 0`); a fourth production at that moment would block on `wait(empty)` until the consumer frees a slot in step ٤.

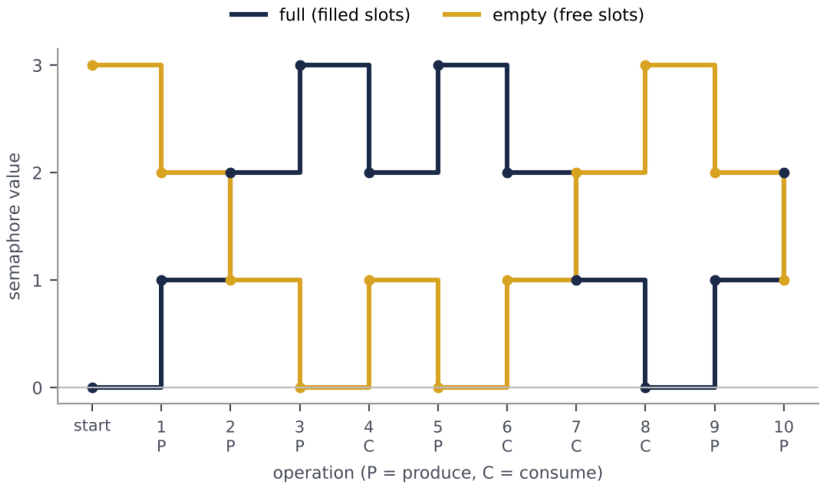


Figure 7 — Values of the empty and full semaphores (buffer capacity 3)

Analysis and Pitfalls

Mutual exclusion: with initial value ١ and matched wait/signal pairs at most one process passes. **Deadlock:** if the producer swaps `wait(mutex)` and `wait(empty)`, it may sleep on `empty` while holding `mutex`, and the consumer can never obtain `mutex`. **Programmer error:** a forgotten `signal` or an extra `wait` leads to deadlock or broken exclusion, and the compiler cannot help because the calls are scattered. **Priority inversion:** a low-priority process holding a semaphore can stall a high-priority one (as in the ١٩٩٧ Mars Pathfinder case).

Aspect	Strengths	Weaknesses
CPU	Blocking: waiting process sleeps, no CPU waste	Context-switch cost for very short critical sections
Generality	Mutual exclusion and signalling; N processes	Low-level; easy to misuse (order, forgotten signal)
Fairness	Strong semaphores give bounded waiting	Weak semaphores may starve
Support	Provided by all major OSs (POSIX, Windows)	Needs OS and atomic hardware support

٥. Monitors

Definition and Idea

Despite their power, semaphores are error-prone. In ١٩٧٤ C. A. R. Hoare, building on Brinch Hansen's work, introduced the **monitor** as a high-level construct. A monitor is an abstract data type that bundles shared data, the procedures operating on it and the synchronization lock. The main rule: **at most one thread is active inside the monitor at any time**. The lock is acquired automatically on entering a procedure and released on leaving, so the programmer cannot forget it.

For conditional synchronization (e.g. "wait until the buffer is not full") monitors use **condition variables**. `wait(c)` puts the thread to sleep in the condition queue and releases the monitor lock at the same time; `signal(c)` wakes one waiting thread. Unlike a semaphore, a condition variable has no memory: a signal with no waiter is lost.

Two Signalling Styles: Hoare and Mesa

Feature	Hoare style	Mesa style
signal	Signaller yields the monitor to the woken thread at once	Signaller continues; woken thread only becomes ready
Condition test	`if` is sufficient (condition still true)	Must use `while` (condition may have changed)
Cost	More context switches	Cheaper; used in practice
Implementations	Textbooks, theory	Java, C#, POSIX threads, Python

Pseudocode

Pseudocode ٥ — Bounded-buffer monitor

```
monitor BoundedBuffer
  var buf[N], in = 0, out = 0, count = 0
  condition notFull, notEmpty

  procedure put(item)
    while count = N do wait(notFull)      // Mesa style: re-test
    buf[in] <- item;  in <- (in + 1) mod N;  count <- count + 1
    signal(notEmpty)

  procedure get() returns item
    while count = 0 do wait(notEmpty)
    item <- buf[out]; out <- (out + 1) mod N; count <- count - 1
    signal(notFull)
    return item
end monitor
// the monitor lock is acquired on entry and released on exit
// of every procedure (and while waiting) automatically
```

C and Java Implementations

C has no monitors, but one can be built from a `pthread_mutex_t` and some `pthread_cond_t` variables. In Java every object is intrinsically a monitor: `synchronized` acquires the lock, and `wait()` / `notifyAll()` act as the condition variable. The C version was tested with one producer, one consumer and ١٠٠,٠٠٠ items; the Java version follows the standard idiom.

Listing ٤ — Bounded-buffer monitor with pthreads (C)

```
#include <pthread.h>
```

```
#define N 8
/* ---- monitor "BoundedBuffer" : all data + lock + conditions ---- */
typedef struct {
    int buf[N];
    int in, out, count;
    pthread_mutex_t lock;         /* the monitor's implicit mutex  */
    pthread_cond_t  not_full;     /* condition variable            */
    pthread_cond_t  not_empty;    /* condition variable            */
} Monitor;

Monitor m = { .lock = PTHREAD_MUTEX_INITIALIZER,
              .not_full  = PTHREAD_COND_INITIALIZER,
              .not_empty = PTHREAD_COND_INITIALIZER };

void mon_put(int item) {                    /* monitor procedure */
    pthread_mutex_lock(&m.lock);            /* enter monitor     */
    while (m.count == N)                    /* Mesa style: while */
        pthread_cond_wait(&m.not_full, &m.lock);
    m.buf[m.in] = item;  m.in = (m.in + 1) % N;  m.count++;
    pthread_cond_signal(&m.not_empty);
    pthread_mutex_unlock(&m.lock);          /* leave monitor     */
}

int mon_get(void) {                         /* monitor procedure */
    pthread_mutex_lock(&m.lock);
    while (m.count == 0)
        pthread_cond_wait(&m.not_empty, &m.lock);
    int item = m.buf[m.out];  m.out = (m.out + 1) % N;  m.count--;
    pthread_cond_signal(&m.not_full);
    pthread_mutex_unlock(&m.lock);
    return item;
}
```

Listing ٥ — Bounded-buffer monitor in Java

```
public class BoundedBuffer {
    private final int[] buf;
    private int in = 0, out = 0, count = 0;

    public BoundedBuffer(int capacity) { buf = new int[capacity]; }

    public synchronized void put(int item) throws InterruptedException {
        while (count == buf.length)          // Mesa semantics: always re-test
            wait();
        buf[in] = item;  in = (in + 1) % buf.length;  count++;
        notifyAll();                          // wake up waiting threads
    }

    public synchronized int get() throws InterruptedException {
        while (count == 0)
            wait();
        int item = buf[out];  out = (out + 1) % buf.length;  count--;
        notifyAll();
        return item;
    }
}
```

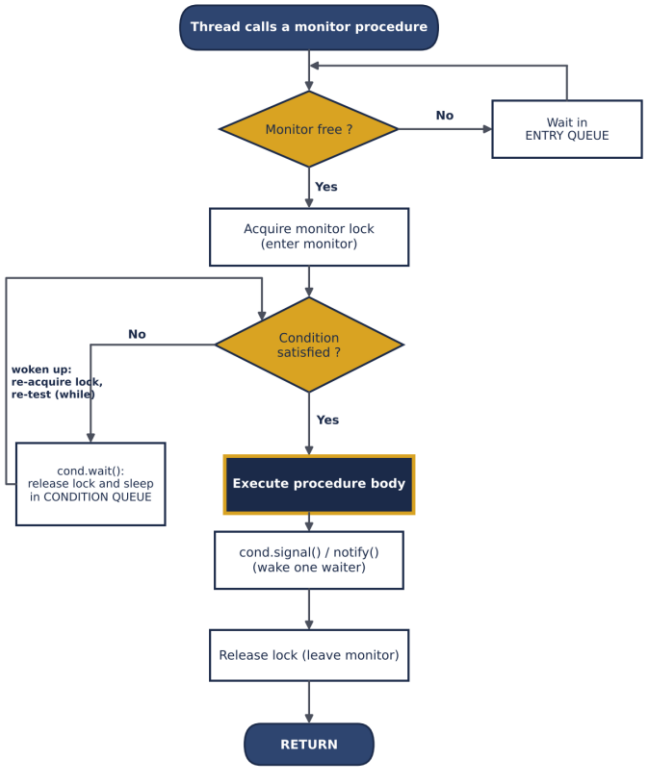


Figure 8 — Flowchart of executing a monitor procedure

Analysis

Mutual exclusion is guaranteed structurally; no user code is needed to lock. **Queues:** threads have an entry queue and one queue per condition variable; bounded waiting depends on the scheduling policy (FIFO queues). **Deadlock:** nested monitor calls or a `wait` without a matching `signal` can deadlock. **Pitfall:** in Mesa style the `while` loop is mandatory — using `if` causes errors through spurious wakeups.

Aspect	Strengths	Weaknesses
Safety	Lock handled by the language; hard to misuse	Requires language/runtime support (not in plain C)
Modularity	Data + synchronization encapsulated in one unit	Nested calls can deadlock
Conditions	Condition variables express complex waits clearly	Signal semantics (Hoare vs Mesa) confuse beginners
Performance	Blocking, no busy waiting	Coarse lock: one thread at a time inside the whole monitor

٦. Readers–Writers

Problem Statement

The Readers–Writers problem was posed by Courtois, Heymans and Parnas in ١٩٧١. Several "reader" processes only read shared data and several "writer" processes modify it. Rules: (١) many readers may read simultaneously; (٢) a writer needs exclusive access — no readers and no other writer. It models databases, caches and file systems, where reads vastly outnumber writes and full exclusion (like a plain mutex) wastes performance.

> **Note**
> Readers–Writers is a synchronization *problem*, not a primitive like the semaphore. Because its standard solutions are textbook algorithms and appear in libraries as read/write locks (RW-locks), it is compared here alongside the other algorithms.

The Three Variants

Variant	Rule	Consequence
First (readers preference)	No reader waits unless a writer already holds the data	Writers may starve
Second (writers preference)	Once a writer waits, no new reader may start	Readers may starve
Third (fair)	Requests served in arrival order (turnstile / queue)	No starvation, slightly lower read concurrency

Pseudocode

Pseudocode ٦ — First readers–writers problem (readers preference)

```
 1  shared: semaphore mutex = 1, rw_mutex = 1
 2          int read_count = 0
 3
 4  writer:                                reader:
 5    wait(rw_mutex)                         wait(mutex)
 6    ----- WRITE -----                      read_count <- read_count + 1
 7    signal(rw_mutex)                       if read_count = 1 then wait(rw_mutex)
 8                                           signal(mutex)
 9                                           ----- READ -----
10                                           wait(mutex)
11                                           read_count <- read_count - 1
12                                           if read_count = 0 then signal(rw_mutex)
13                                           signal(mutex)
```

Pseudocode ٧ — Fair version with a turnstile

```
 1  shared: semaphore mutex = 1, rw_mutex = 1, turnstile = 1
 2          int read_count = 0
 3
 4  writer:                                reader:
 5    wait(turnstile)                        wait(turnstile)    // pass the gate
 6    wait(rw_mutex)                         signal(turnstile)
 7    ----- WRITE -----                      (same entry code as before)
 8    signal(turnstile)                      ----- READ -----
 9    signal(rw_mutex)                       (same exit code as before)
10  // a waiting writer holds the turnstile, so no new reader can pass it
```

In the first version the "first reader" locks the writers out and the "last reader" lets them in; the counter `read_count` is protected by `mutex`. In the fair version a waiting writer closes the `turnstile` and new readers queue behind it, so the writer is served as soon as the current readers leave.

C Implementation

The code below provides both versions using POSIX semaphores. In the test, three readers and two writers ran concurrently; writers incremented two array elements with a pause in between, and readers checked that the two were always equal. No violation was seen in either version.

Listing ٦ — Readers–Writers (basic and fair)

```
#include <semaphore.h>

/* ---------- Readers-Writers, first problem (readers preference) ---------- */
sem_t rw_mutex;                  /* init 1 : exclusive access to the data   */
sem_t mutex;                     /* init 1 : protects read_count            */
int   read_count = 0;

void writer_enter(void) { sem_wait(&rw_mutex); }
void writer_exit (void) { sem_post(&rw_mutex); }

void reader_enter(void) {
    sem_wait(&mutex);
    if (++read_count == 1)       /* first reader locks out the writers      */
        sem_wait(&rw_mutex);
    sem_post(&mutex);
}
void reader_exit(void) {
    sem_wait(&mutex);
    if (--read_count == 0)       /* last reader lets the writers in         */
        sem_post(&rw_mutex);
    sem_post(&mutex);
}

/* ---------- Fair version: a turnstile stops new readers once a writer waits ---------- */
sem_t turnstile;                 /* init 1 */

void fair_writer_enter(void) { sem_wait(&turnstile); sem_wait(&rw_mutex); }
void fair_writer_exit (void) { sem_post(&turnstile); sem_post(&rw_mutex); }

void fair_reader_enter(void) {
    sem_wait(&turnstile);  sem_post(&turnstile);   /* pass through the gate */
    reader_enter();
}
void fair_reader_exit(void) { reader_exit(); }
```

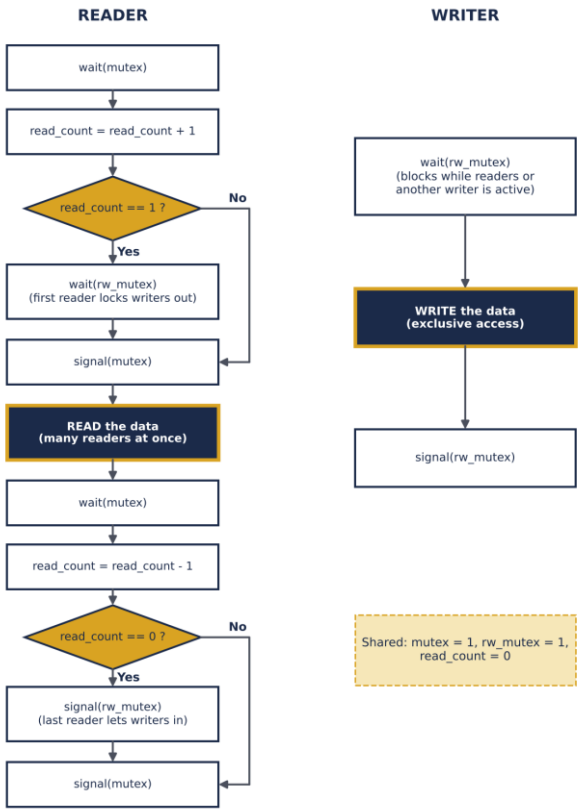


Figure 9 — Flowchart of reader and writer (first variant)

Timeline and Starvation

Figure ١٠ shows a run of the first variant. Writer W١ requests at time ٢, but since reader R٣ can still join the active readers at time ٤, W١ waits until time ٧. If new readers keep arriving, W١ never gets its turn — this is writer starvation in the first variant.

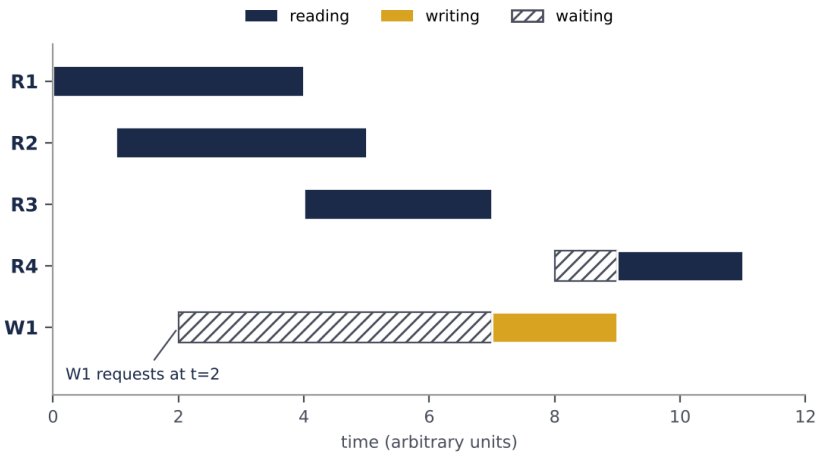


Figure 10 — Gantt chart: writer starvation in the first variant

Analysis

Mutual exclusion: a writer excludes everybody via `rw_mutex`; readers share access with each other but not with writers. **Concurrency:** up to N readers read simultaneously — a property none of the other four algorithms offers.

Starvation: variant ١ starves writers, variant ٢ starves readers, variant ٣ starves neither. **Use:** `pthread_rwlock_t` in POSIX and `ReentrantReadWriteLock` in Java provide this pattern ready-made.

Aspect	Strengths	Weaknesses
Throughput	Parallel reads: high throughput for read-heavy loads	If writes are frequent, a plain mutex is simpler and as fast
Fairness	Fair variant avoids starvation	Basic variants starve writers or readers
Complexity	Built from ٢-٣ semaphores and a counter	More states and subtle ordering than a simple lock

٧. Comprehensive Comparison

This chapter compares the five solutions from two points of view: correctness and behaviour (Table ٣) and engineering and efficiency (Table ٤). A qualitative scoring follows in Table ٥, together with comparison charts. Quantitative values such as lines of code come from the tested listings of this article.

Correctness and Behaviour

Table ٣ — Correctness and behaviour properties (mutual exclusion, progress, bounded waiting, …)

Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Mutual exclusion	Yes	Yes	Yes	Yes (structural)	Writers exclusive, readers shared
Progress	Yes	Yes	Yes	Yes	Yes
Bounded waiting	Yes (≤ ١ turn)	Yes (≤ ١ turn)	Only strong (FIFO) semaphores	Depends on queue policy	Variants ١/٢ no, fair variant yes
Starvation	No	No	Possible (weak semaphore)	Possible (Mesa barging)	Possible (writer or reader)
Deadlock	No	No	Possible (wrong wait order)	Possible (nested monitors)	Not by itself; possible with other locks
Busy waiting	Yes	Yes	No (blocking)	No	No
Processes	٢	٢	N	N	N readers + M writers
Concurrency inside CS	No	No	Counting: up to k threads	No (one thread)	Yes: parallel readers
Conditional signalling	No	No	Yes (wait / signal)	Yes (condition variables)	Via semaphores / monitors
Priority inversion	Possible while spinning	Possible while spinning	Possible	Possible	Possible

Engineering and Efficiency

Table ٤ — Engineering parameters: hardware, CPU, scalability, complexity and use

Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Level	Software algorithm	Software algorithm	OS primitive	Language construct	Synchronization pattern
Support needed	Atomic load/store + barriers	Atomic load/store + barriers	OS scheduler + atomic ops	Language or mutex/cond library	Semaphores / monitor / RW-lock
CPU while waiting	High (spins)	High (spins)	Low (sleeps)	Low (sleeps)	Low (sleeps)
Scalability	Poor (٢N+١ vars)	Poor (filter: ٢N−١)	Good (١ var)	Good	Good (٣ vars)
Lines of code (our listings)	١٨	١٢	٢٦	٢٨ (C)	٢٥
Sync variables (٢ procs)	٣	٣	٣	٣ (lock + ٢ conditions)	٣
Ease of verification	Hard	Medium (short proof)	Medium (error-prone)	Easy (structured)	Medium
Multicore reliability	Needs fences	Needs fences	Reliable	Reliable	Reliable
Modern use	Teaching, theory	Teaching, model checking	Kernels, POSIX, Windows	Java, C#, Python	Databases, caches, file systems

Qualitative Scoring

Table ٥ gives each criterion a score from ١ (poor) to ٥ (excellent). The scores are the **author's qualitative judgement** based on the previous tables, not measurements. A higher "independence from OS and hardware" means less support is needed. All criteria have equal weight; the average is in the last row.

Table ٥ — Qualitative scores of the solutions (١–٥)

Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Mutual-exclusion safety	٥	٥	٥	٥	٥
Fairness / bounded waiting	٤	٥	٤	٤	٢
CPU efficiency	١	١	٤	٤	٤
Scalability	١	١	٥	٥	٤
Ease of programming	١	٢	٣	٥	٣
Independence from OS / HW	٤	٤	٢	١	٢
Average	٢,٦٧	٣,٠٠	٣,٨٣	٤,٠٠	٣,٣٣

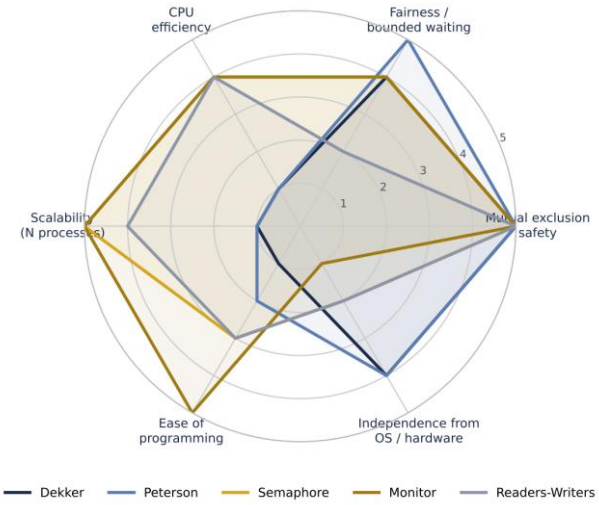


Figure 11 — Radar chart comparing the six criteria

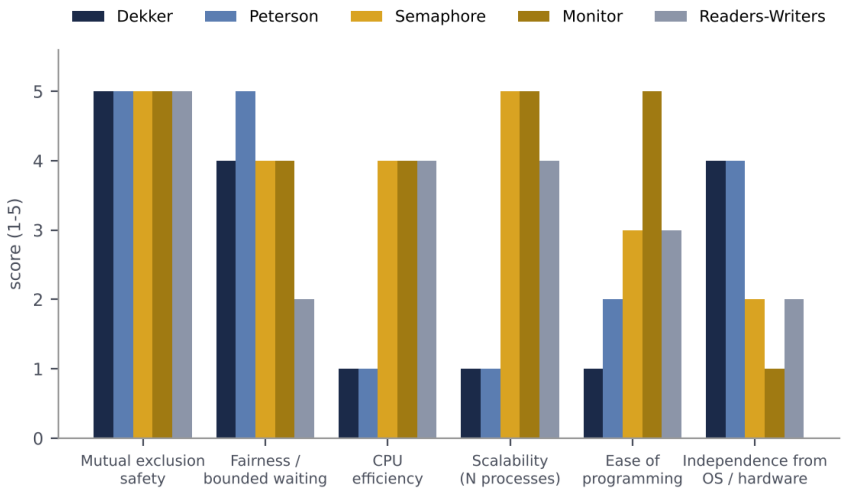


Figure 12 — Grouped bar chart of the score per criterion

The radar chart shows that Dekker and Peterson are strong only in safety and independence from the OS, while semaphores and monitors win in efficiency and scalability. Monitors are the easiest to program but depend on language support. The basic Readers–Writers solution is weaker in fairness but provides parallel reads.

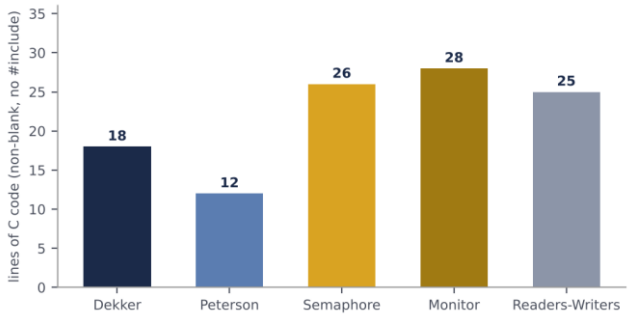


Figure 13 — Implementation size (effective C lines)

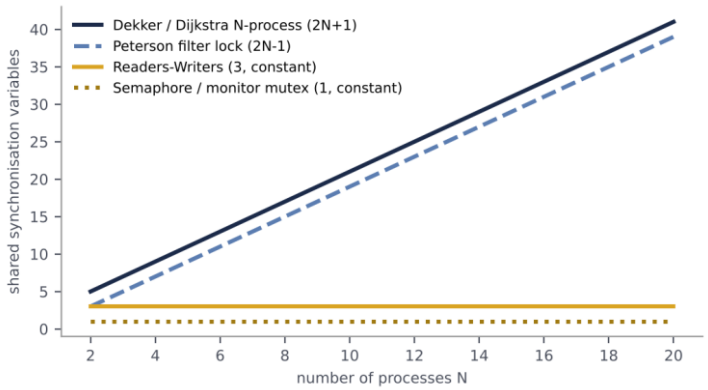


Figure 14 — Shared synchronization variables versus number of processes

Figure ١٤ shows scalability. Generalising software algorithms to N processes needs a linear number of variables (about ٢N) and every entry must read all of them, whereas semaphores and monitors use a constant number of variables for any number of processes and put threads to sleep instead of spinning.

٨. Worked Examples and Sample Questions

For each of the five algorithms we give one worked example (a step-by-step trace) and three sample questions (two multiple-choice and one descriptive) with answers. In all traces the scheduler is assumed to execute one statement of one process at a time.

Dekker

Example ١: simultaneous arrival

Initial state: `wants = [F, F]`, `turn = 0`. Both processes request entry at the same time. Determine which one enters.

Step	Process	Action	wants[٠], wants[١]	turn
١	P٠	wants[٠] = true	T, F	٠
٢	P١	wants[١] = true	T, T	٠
٣	P٠	sees wants[١]=T; turn = ٠ = i, so no back-off; keeps looping	T, T	٠
٤	P١	sees wants[٠]=T; turn ≠ ١ → back off: wants[١] = false; spins	T, F	٠
٥	P٠	sees wants[١]=F → leaves loop → enters CS	T, F	٠
٦	P٠	exit: turn = ١; wants[٠] = false	F, F	١
٧	P١	turn = ١ → wants[١] = true; wants[٠]=F → enters CS	F, T	١

Result: P٠ enters first because `turn = 0` gives it priority; P١ backs off and enters right after P٠ leaves. Mutual exclusion and bounded waiting hold.

Question ١. What is the purpose of the back-off (`wants[i] = false`) in Dekker's algorithm?

(a) faster execution (b) avoid deadlock when both flags are up and the turn is the rival's (c) free memory (d) switch the turn

Answer: (b). The process without priority lowers its flag so that its rival can pass.

Question ٢. For how many processes does the original Dekker algorithm work?

(a) ١ (b) ٢ (c) N (d) unlimited

Answer: (b). Dijkstra presented the N-process version separately.

Question ٣. If `turn = j` is removed from the exit section, which requirement is violated? Explain.

Answer: Bounded waiting. If `turn` stays ٠ forever, P١ must back off in every conflict while P٠, requesting repeatedly, can always win, so P١ starves. Mutual exclusion still holds.

Peterson

Example ٢: trace and statement order

Initial state: `flag = [F, F]`. P٠ and P١ interleave. In each loop condition `j` is the rival.

Step	Process	Action	flag[٠], flag[١]	turn
١	P٠	flag[٠] = true	T, F	?
٢	P١	flag[١] = true	T, T	?
٣	P١	turn = ٠ (gives the turn to P٠)	T, T	٠
٤	P٠	turn = ١ (gives the turn to P١)	T, T	١
٥	P٠	flag[١] ∧ turn = ١ → TRUE → P٠ waits	T, T	١
٦	P١	flag[٠] ∧ turn = ٠ → FALSE → P١ enters CS	T, T	١
٧	P١	exit: flag[١] = false	T, F	١
٨	P٠	flag[١] = F → condition FALSE → P٠ enters CS	T, F	١

Result: the last process to write `turn` (P٠) waits and the other (P١) enters.

Question ٤. In Peterson's algorithm, if two processes arrive simultaneously, which one waits?

(a) the first to raise its flag (b) the last to write `turn` (c) the lower-numbered process (d) both

Answer: (b).

Question ٥. Which is Peterson's waiting condition?

(a) `flag[j] || turn == j` (b) `flag[j] && turn == j` (c) `!flag[j] && turn == i` (d) `flag[i] && turn == i`

Answer: (b): a process waits only if the rival is interested and the turn is the rival's.

Question ٦. Swap the two entry statements (`turn = j` first, then `flag[i] = true`). Show that mutual exclusion is violated.

Answer: Trace: P٠: `turn = 1`; P١: `turn = 0`; P١: `flag[1] = true`, and since `flag[0] = false` P١ enters the CS; then P٠: `flag[0] = true`, and since `turn = 0` the test `turn == 1` is false so P٠ enters too. Both are inside the CS.

Semaphore

Example ٣: bounded buffer of capacity ٢

Initial values: `empty = 2`, `full = 0`. In the blocking implementation a semaphore can become negative (its absolute value is the number of waiting processes).

Step	Action	empty	full	Note
١	Producer: wait(empty) … signal(full) (item A)	١	١	A stored
٢	Producer: item B	٠	٢	buffer full
٣	Producer: wait(empty) for item C	−١	٢	producer BLOCKS (queue: Prod)
٤	Consumer: wait(full) … removes A	−١	١	one slot freed
٥	Consumer: signal(empty)	٠	١	value ≤ ٠ → wakes producer
٦	Producer resumes, stores C, signal(full)	٠	٢	buffer full again

Result: `empty + full` always equals the capacity (٢), except while a process is blocked. The producer sleeps in step ٣ without wasting CPU and is woken in step ٥.

Question ٧. What is the initial value of the `mutex` semaphore used for mutual exclusion?

(a) ٠ (b) ١ (c) N (d) ٢

Answer: (b); the value ١ means the critical section is free, and only one process passes `wait`.

Question ٨. In the blocking implementation, what does a semaphore value of −٣ mean?

(a) three free resources (b) three processes blocked in the queue (c) an error (d) three extra signals

Answer: (b).

Question ٩. Why does swapping `wait(mutex)` and `wait(empty)` in the producer create a deadlock?

Answer: If the buffer is full, the producer takes `mutex` and then sleeps on `wait(empty)` while still holding `mutex`. The consumer needs `wait(mutex)` to free a slot and can never get it, so both wait forever.

Monitor

Example ٤: why `while`? (Mesa style)

The buffer is empty. Consider consumers C١ and C٢ and a producer P:

Step	Event	State
١	C١ calls get(): count = ٠ → wait(notEmpty); releases the lock	C١ sleeps in the notEmpty queue
٢	P calls put(١٠): count = ١; signal(notEmpty)	C١ becomes READY but P still holds the lock
٣	P leaves the monitor	lock is free
٤	C٢ calls get() and acquires the lock first: takes the item, count = ٠	buffer empty again
٥	C١ re-acquires the lock and re-tests: count = ٠ → TRUE	with `while`: C١ waits again (correct); with `if`: C١ would read an empty buffer (BUG)

Result: in Mesa style, being woken up does not guarantee that the condition still holds; therefore the condition must be re-tested in a `while` loop.

Question ١٠. Why is `while` used instead of `if` in a Mesa-style monitor?

(a) speed (b) the condition may change between the signal and resumption (c) compiler requirement (d) avoid stack overflow

Answer: (b).

Question ١١. What is the difference between a semaphore `signal` and a condition-variable `signal` when nobody is waiting?

(a) none (b) the semaphore increments and remembers; the condition variable loses the signal (c) both raise errors (d) the semaphore loses it

Answer: (b).

Question ١٢. In this article's Java code, why is `notifyAll()` used instead of `notify()`?

Answer: Producers and consumers sleep on one shared wait set. `notify()` may wake the wrong kind (e.g. a producer while the buffer is still full); that thread sleeps again and the wake-up never reaches the real consumer, causing deadlock. `notifyAll()` wakes everyone and each thread re-tests its own condition.

Readers–Writers

Example ٥: tracing the first variant

Events: R١ arrives at time ٠, R٢ at ١, writer W١ at ٢, R٣ at ٣; R١ leaves at ٤, R٢ at ٥ and R٣ at ٦.

Time	Event	read_count	rw_mutex	Note
٠	R١ enters: first reader → wait(rw_mutex)	١	٠ (taken)	R١ reads
١	R٢ enters	٢	٠	R١ and R٢ read together
٢	W١: wait(rw_mutex)	٢	٠	W١ BLOCKS
٣	R٣ enters (readers preference)	٣	٠	joins the readers
٤	R١ leaves	٢	٠	—
٥	R٢ leaves	١	٠	—
٦	R٣ leaves: last reader → signal(rw_mutex)	٠	١ → W١	W١ wakes and writes

Result: R٣ overtook the waiting writer W١; with more arriving readers W١ would starve. In the fair version R٣ would have waited at the `turnstile`.

Question ١٣. In the first Readers–Writers variant, which group may starve?

(a) readers (b) writers (c) neither (d) both

Answer: (b).

Question ١٤. What does the first reader do in the first variant?

(a) runs `signal(mutex)` (b) locks writers out with `wait(rw_mutex)` (c) waits for the writer (d) resets `read_count`

Answer: (b).

Question ١٥. How does the `turnstile` remove writer starvation?

Answer: The writer takes `turnstile` before `rw_mutex` and holds it until the write ends. A new reader must pass the `turnstile` and is held behind the writer, so only the current readers finish and the writer enters right after them.

٩. Conclusion and Selection Guide

The study shows that none of the five solutions wins on every criterion. Dekker and Peterson prove that mutual exclusion is possible with plain reads and writes, but because of busy waiting and the two-process limit they are mainly used for teaching and verification. The semaphore is the general-purpose OS tool providing both exclusion and inter-process coordination, but it is error-prone. The monitor, by hiding the lock inside a structure, is the safest and most readable choice in languages that support it. Readers–Writers shows that when access is mostly reading, accepting parallel reads greatly improves performance, provided starvation is managed.

Table ٦ — Recommendation by scenario

Scenario	Recommendation	Reason
Teaching mutual exclusion without OS support	Peterson (or Dekker)	Shortest correct software algorithm
Many processes in an OS kernel or C/POSIX	Semaphore	Blocking, N processes, in every OS
Java / C# / Python with complex conditions	Monitor	Lock handled by the language
Read-mostly data (cache, database)	Readers–Writers (fair)	Parallel reads, no starvation

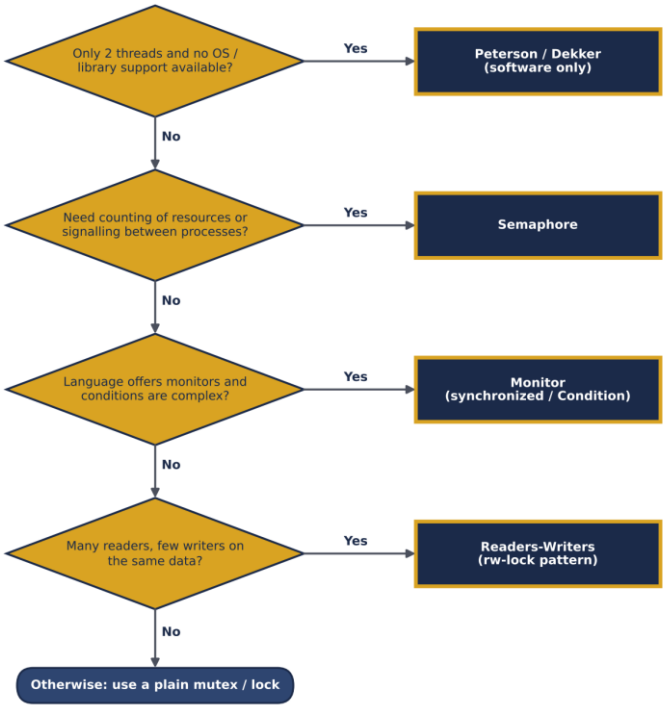


Figure 15 — Decision flowchart for choosing a synchronization mechanism

Final message

In practice programmers use ready-made library locks (`pthread_mutex`, `synchronized`, `ReentrantReadWriteLock`) instead of writing Dekker or Peterson; yet knowing these five algorithms explains why those locks are correct and what the common failures (deadlock, starvation, priority inversion) are.

References

[١] E. W. Dijkstra, “Solution of a Problem in Concurrent Programming Control,” Communications of the ACM, ٨(٩), p. ٥٦٩, ١٩٦٥.

[٢] E. W. Dijkstra, “Cooperating Sequential Processes,” Technical Report EWD-١٢٣, Technische Hogeschool Eindhoven, ١٩٦٥.

[٣] G. L. Peterson, “Myths About the Mutual Exclusion Problem,” Information Processing Letters, ١٢(٣), pp. ١١٥–١١٦, ١٩٨١.

[٤] C. A. R. Hoare, “Monitors: An Operating System Structuring Concept,” Communications of the ACM, ١٧(١٠), pp. ٥٤٩–٥٥٧, ١٩٧٤.

[٥] P. J. Courtois, F. Heymans, D. L. Parnas, “Concurrent Control with ‘Readers’ and ‘Writers’,” Communications of the ACM, ١٤(١٠), pp. ٦٦٧–٦٦٨, ١٩٧١.

[٦] B. W. Lampson, D. D. Redell, “Experience with Processes and Monitors in Mesa,” Communications of the ACM, ٢٣(٢), pp. ١٠٥–١١٧, ١٩٨٠.

[٧] A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, ١٠th ed., Wiley, ٢٠١٨.

[٨] A. S. Tanenbaum, H. Bos, Modern Operating Systems, ٤th ed., Pearson, ٢٠١٤.

[٩] M. Herlihy, N. Shavit, The Art of Multiprocessor Programming, ٢nd ed., Morgan Kaufmann, ٢٠٢٠.

[١٠] A. B. Downey, The Little Book of Semaphores, Green Tea Press, ٢٠١٦.